

Pflichtenheft und Agile Dokumentation

Projekt	Musikplayer zur genrebasierten Wiedergabe (MusicPlayerMVVM)	Klasse	6AAIFT
Autor	Lukas Jaros	Datum	19.05.2026

1. Projektübersicht

Das Ziel des Projekts ist die Entwicklung eines modularen Musikplayers zur genrebasierten Wiedergabe von Songs. Die Anwendung stellt dem Benutzer vordefinierte Playlists bereit und ermöglicht in einer separaten Custom-Playlist das interaktive Hinzufügen und Löschen von Musiktiteln.

Die Software wird architektonisch von der klassischen WPF-Code-Behind-Struktur auf das moderne MVVM-Konzept (Model-View-ViewModel) umgestellt. Dadurch werden Benutzeroberfläche (Design), Programmlogik und die Datenverwaltung strikt voneinander getrennt. Die persistente Datenhaltung wird über Entity Framework Core (EF Core) mittels eines Object-Relational-Mappers (ORM) realisiert, um Typsicherheit, Wartbarkeit und ein sauberes relationales Modell zu garantieren.

2. Benutzeroberfläche (Views & Ergonomie)

Die gesamte Oberfläche wird über entkoppelte XAML-Views ohne C#-Logik im Code-Behind realisiert. Die Steuerung und Navigation erfolgt über ViewModels mittels dynamischer Datenbindung (Data Binding).

- **MainWindow:** Das zentrale Hauptfenster. Es lädt über ein dynamisches ContentControl den aktuellen ViewContext (Home oder Playlist), ohne neue Fenster zu öffnen.
- **HomeView:** Der Startbildschirm der Anwendung. Er enthält ansprechende Genre-Buttons (HipHop, Rock, Pop, Klassik, Jazz, Custom) mit Bild und Text.
- **Genre-Playlists (Views):** Spezifische Oberflächen zur Darstellung der Lieder des ausgewählten Genres (Anzeige von Titel, Interpret und Spieldauer in einer ListView).
- **Steuerungselemente:** Einheitliche, ergonomische Bedienelemente für die Wiedergabe (Play/Pause, Nächster/Vorheriger Titel) und eine stufenlose Lautstärkeregelung.
- **Custom-View:** Ermöglicht das interaktive Erfassen neuer Songs via File-Dialog (MP3/WAV) und das Löschen von Einträgen.
- **Responsivität:** Das Layout ist vollständig flexibel gestaltet. Fixe Fenstergrößen sind im Code verboten. Das Fenster nutzt ResizeMode="CanResize". Durch den Einsatz von Grid-Layouts mit proportionalen Zeilen verbleiben die Steuerungselemente und Slider bei Skalierung des Fensters immer im sichtbaren Bereich.
- **Userführung:** Integration von ToolTips auf allen interaktiven Buttons zur intuitiven Bedienung.

3. Musikverwaltung & Datenstruktur (EF Core)

- **Datenzugriffsschicht:** Die native SQL-Logik (ADO.NET mit SqlCommand und nativer SQL-Verbindungszeichenfolge) wird vollständig durch EF Core ersetzt. Abfragen an die Datenbank erfolgen typsicher über LINQ.
- **Top 5 Songs:** Aus der Datenbank werden pro Genre automatisch die Top 5 Songs geladen.

- **Relationales Datenbankmodell (Code-First):** Die Struktur wird normalisiert und über ein zentrales DbContext-Modell abgebildet:
 - **Tabelle Songs:** Id (Primary Key), Title, Artist, Duration, FilePath, GenreId (Foreign Key)
 - **Tabelle Genres:** Id (Primary Key), Name (HipHop, Rock, Pop, Klassik, Jazz, Custom)
- **Änderungsverfolgung:** Datenänderungen (wie das Hinzufügen/Löschen in der Custom-Playlist) werden über das EF Core ChangeTracking erfasst und über das INotifyPropertyChanged-Event in Echtzeit an die Benutzeroberfläche weitergegeben.
- **Fallback-Serialisierung:** Zur Erfüllung der Ausfallsicherheit verfügt das System über eine umschaltbare Datenspeicherung. Bei Fehlern mit dem Datenbanksystem greift die Anwendung auf eine lokale Datei-Serialisierung (JSON/XML) zurück.

4. Musik-Wiedergabe & Logik

- Direkte Wiedergabe ausgewählter Songs über die Windows-Medienkomponenten (MediaPlayer).
- Nahtlose Navigation (vor und zurück) durch die geladene Songliste.
- Stufenlose Lautstärkeregelung über proportionale Slider-Datenbindung.
- Der Datenaustausch zwischen den Komponenten erfolgt lose gekoppelt über Events (INotifyPropertyChanged).

5. Technische Spezifikationen & Qualitätssicherung

- **Klassendesign (OOP):** Implementierung von mindestens drei eigenen, funktionalen Klassen mit echter Logik und Vererbung im Ordner Models (Song, die abstrakte Basisklasse BasePlaylist und die abgeleitete Klasse MusicPlayer). Unterrichtsvorlagen zählen nicht dazu.
- **Performance:** Die Ladezeit einer Playlist liegt unter einer Sekunde (bei max. 5 Songs).
- **Unterstützte Formate:** MP3, WAV.
- **Robuste Fehlerbehandlung:** Umfassende Absicherung aller Datei- und Datenbankzugriffe über Try-Catch-Blöcke zur Gewährleistung der Absturzsicherheit bei Eingaben und Dateizugriffen.
- **Code-Dokumentation:** Der gesamte Quellcode wird nach dem standardisierten XML-Kommentarsystem (///) dokumentiert. Über das Tool Doxygen wird automatisch eine interaktive HTML-Entwicklerdokumentation generiert.

6. Systemvoraussetzungen

- Betriebssystem: Windows 10 oder höher.
- Laufzeitumgebung: .NET Framework oder .NET Core (.NET 6/8 empfohlen).
- Datenbanksystem: Microsoft SQL Server (LocalDB oder Express).

7. Abnahmekriterien

- Fehlerfreier Anwendungsstart und korrekte Darstellung des Startbildschirms (HomeView).
- Funktionierender Wechsel zwischen den Genres ohne UI-Blockaden.
- Korrekte Bindung der Songdaten an die ListView via MVVM Data Binding.
- Fehlerfreies Abspielen, Pausieren, Hinzufügen und Löschen von Songs.
- Absturzsicherheit: Benutzerfreundliche Fehleranzeigen bei fehlenden Musikdateien oder unterbrochener Datenbankverbindung (Aktivierung des File-Fallbacks).